Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence and Data Science

Name: Krish Makhija	Roll no: 57	Branch: AI&DS
Subject: ASD&D	Batch: T13	Date: 03-09-2026

EXPERIMENT 5: To automate application deployment using GitHub/ Jenkins.

Aim: To automate application deployment using Github/Jenkins.

LO mapped: LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

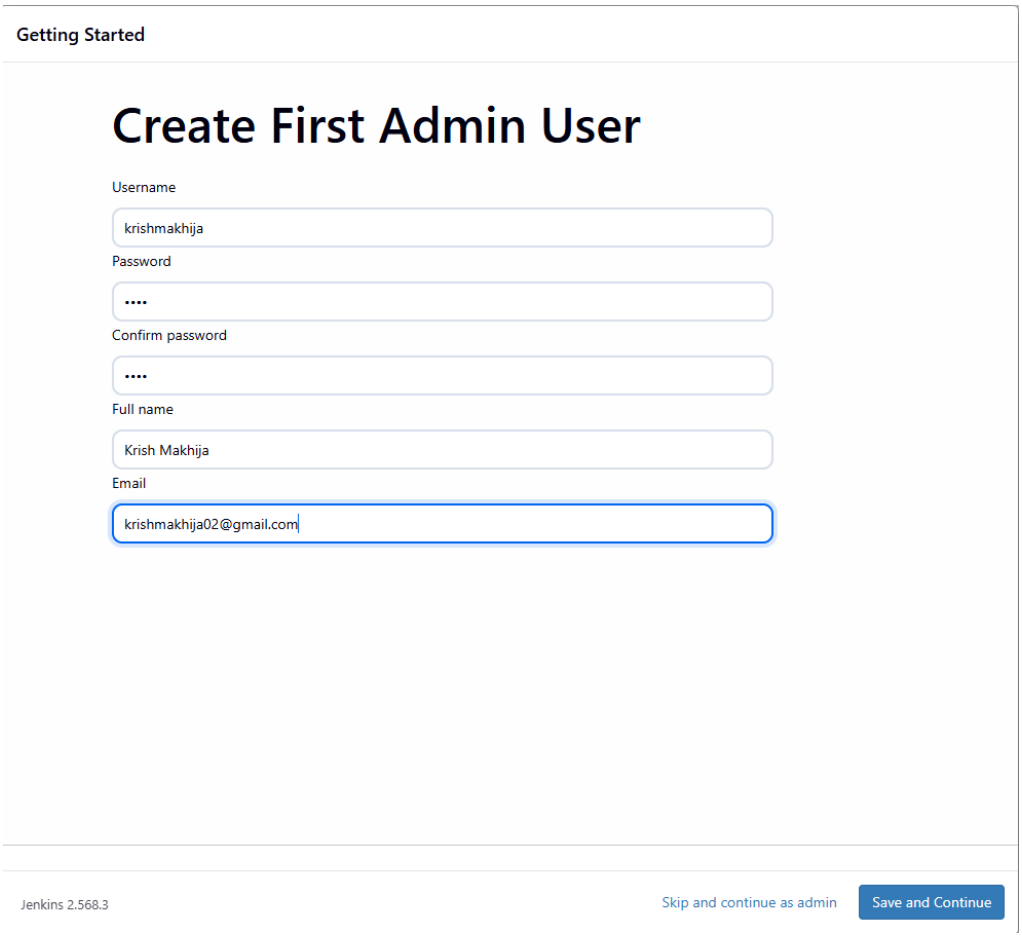
Theory:

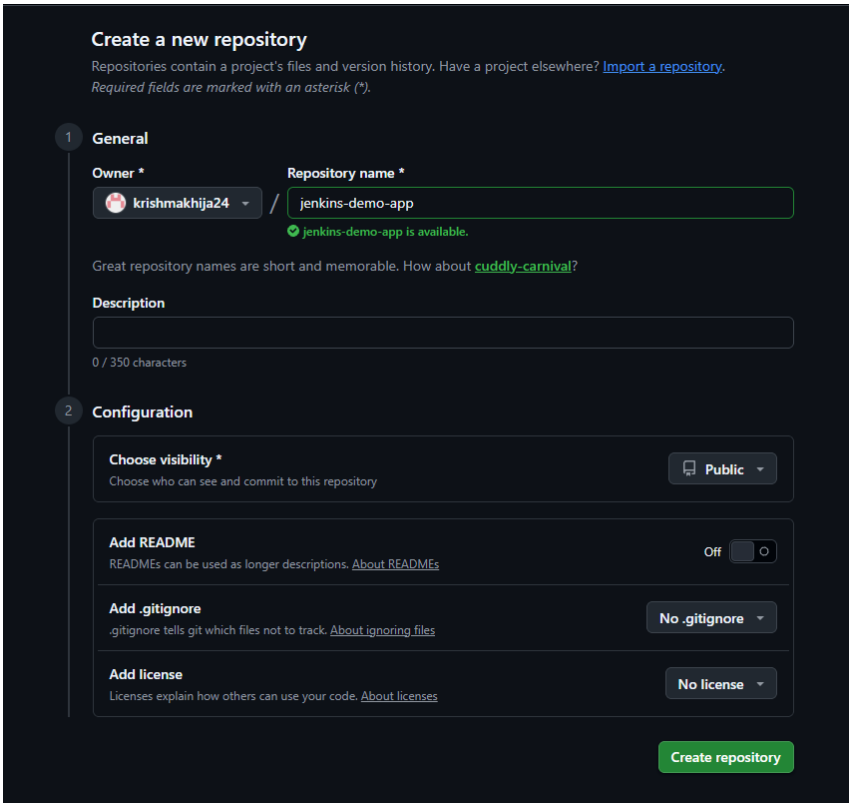
Continuous Integration and Continuous Deployment form the backbone of modern agile workflows by bridging the gap between development, quality assurance, and IT operations. Beyond catching integration conflicts early, CI establishes an automated feedback loop where every commit undergoes systematic validation, preventing the accumulation of technical debt and reducing integration overhead.

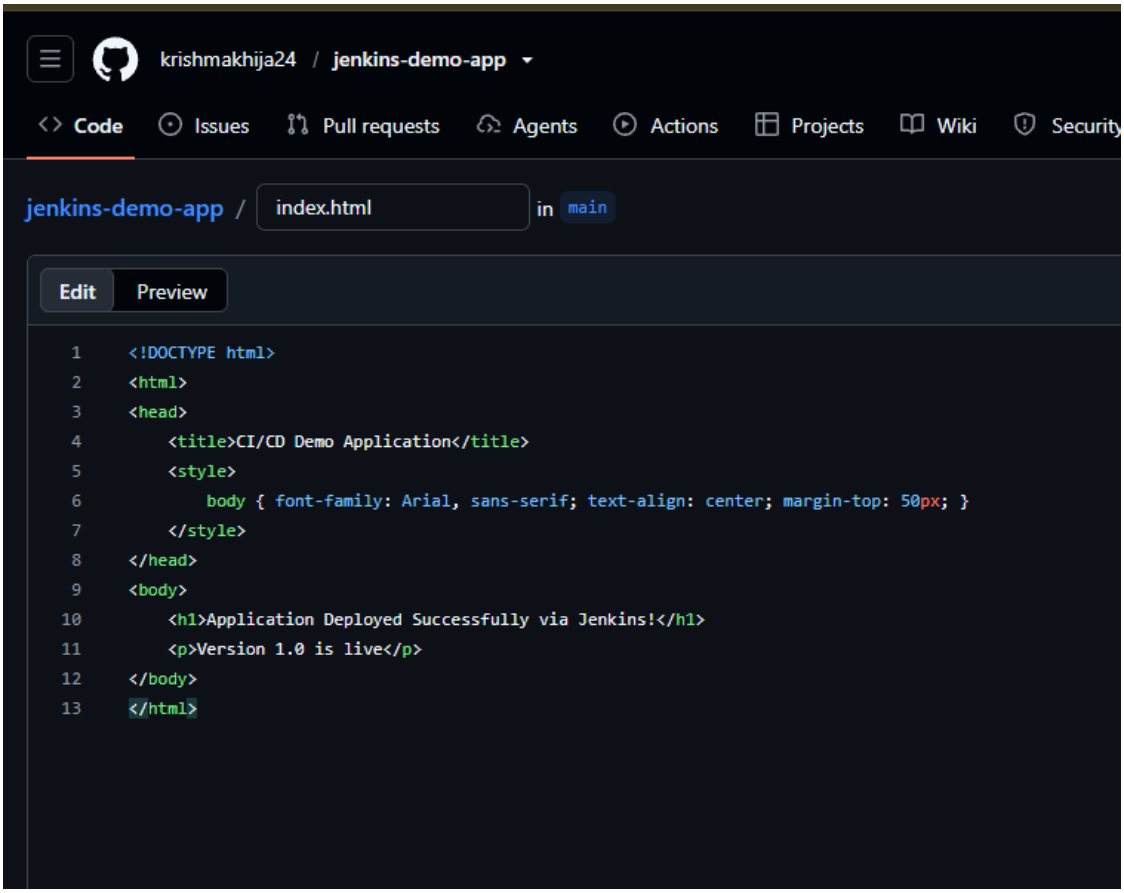
Jenkins facilitates this automation by serving as an extensible orchestration engine equipped with a rich ecosystem of plugins. Within this framework, Pipeline as Code treats the deployment workflow with the exact same rigor as application code. Storing the pipeline definition inside a Jenkinsfile directly alongside the source code ensures traceability, peer review, and simplified auditing across teams.

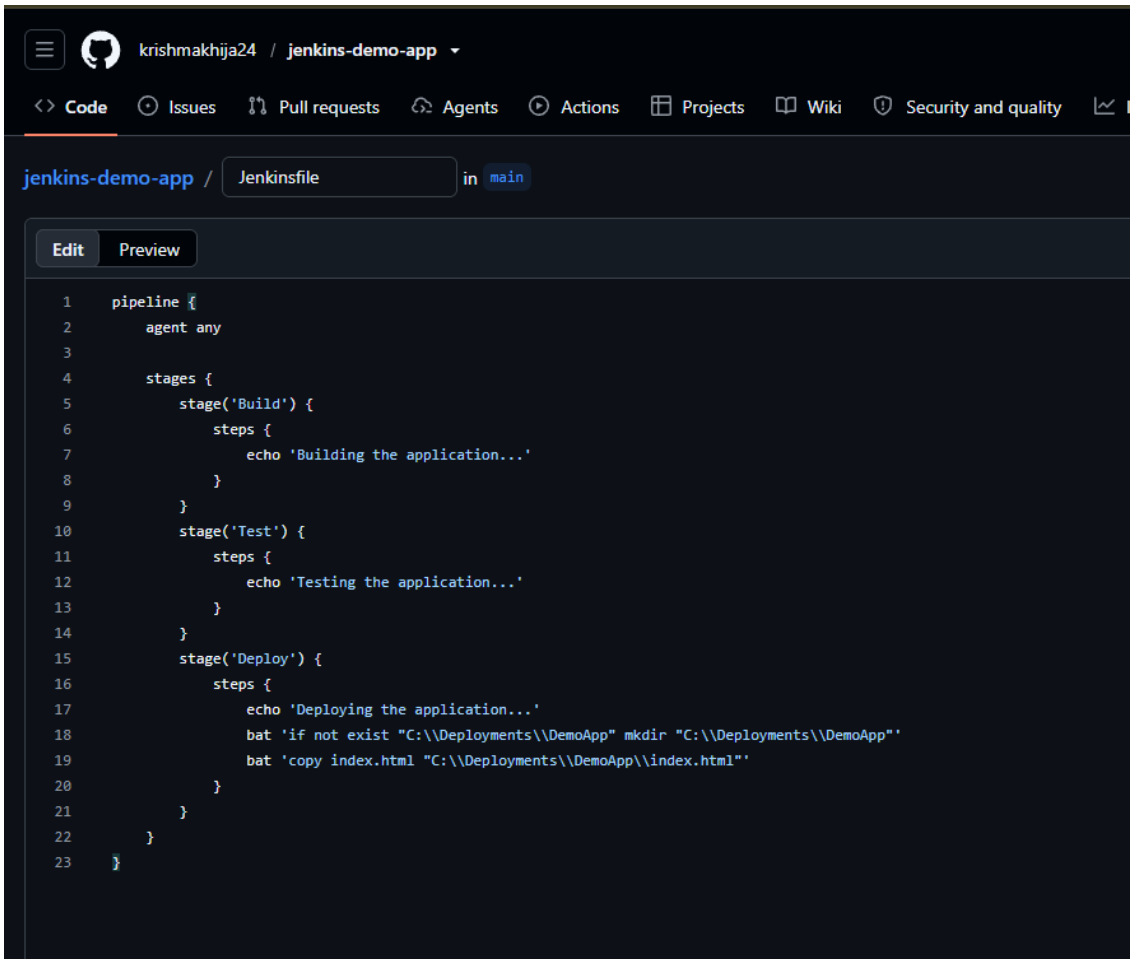
Through declarative pipeline constructs—such as explicit stages, isolated execution blocks via steps, and agent allocation—Jenkins standardizes complex build lifecycles into predictable, repeatable steps. This eliminates environment discrepancies between local machines and production servers, accelerates release velocity, and significantly mitigates the risk of human error during production cutovers.

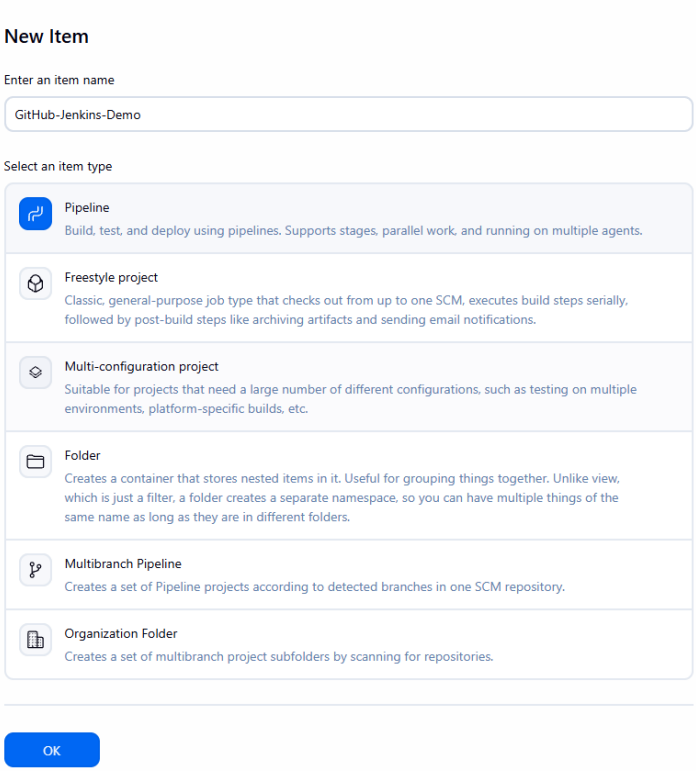
Implementation:

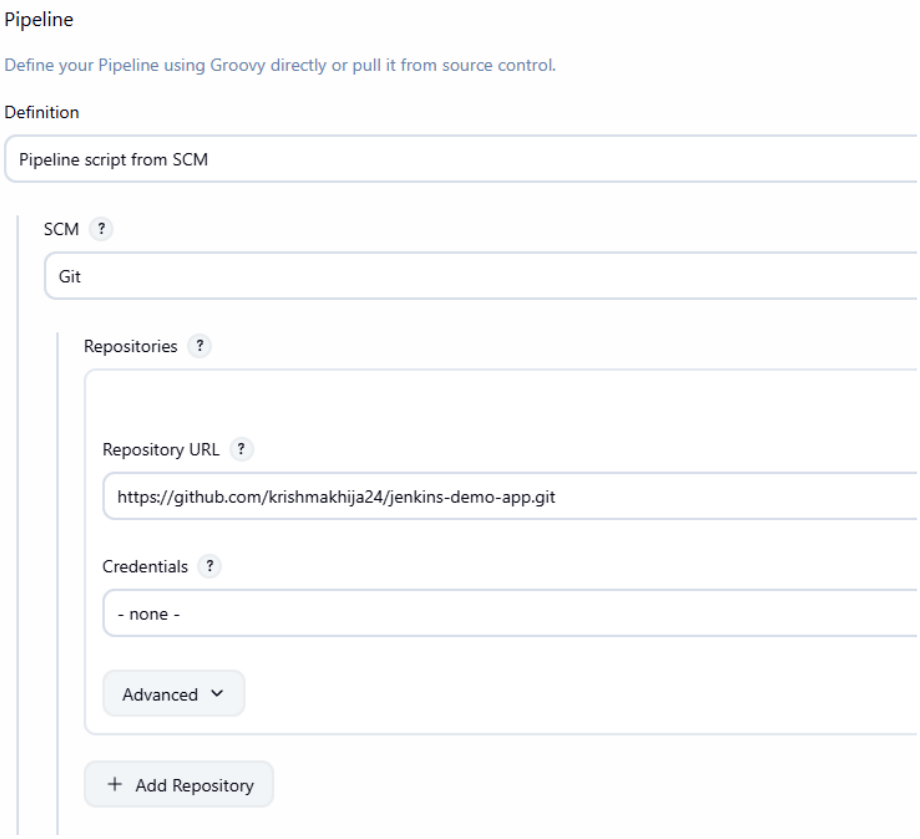


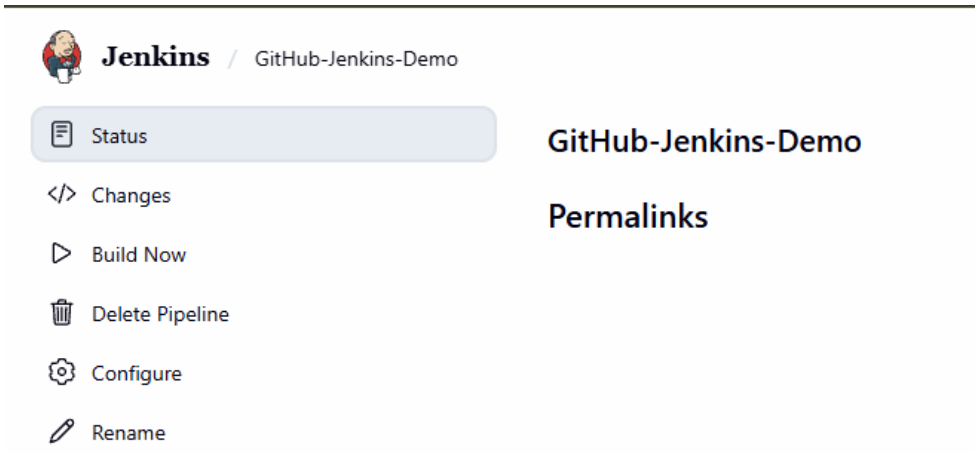

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1
General
Owner *
Repository name *
krishmakhija24
/
jenkins-demo-app
jenkins-demo-app is available.
Great repository names are short and memorable. How about cuddly-carnival?
Description
0 / 350 characters
2
Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
Off
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
Create repository

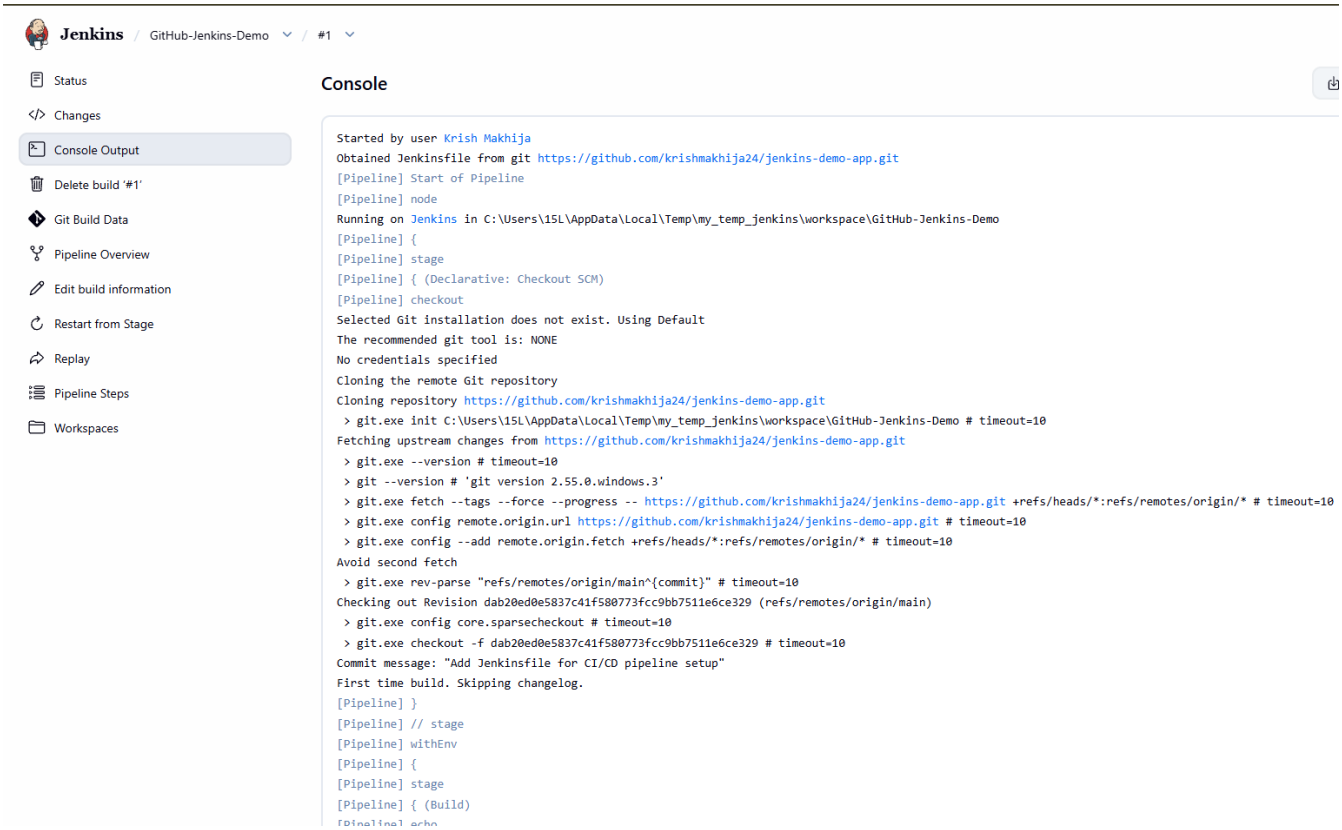

krishmakhija24 / jenkins-demo-app
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security
jenkins-demo-app / index.html in main
Edit
Preview
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>CI/CD Demo Application</title>
5 <style>
6 body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; }
7 </style>
8 </head>
9 <body>
10 <h1>Application Deployed Successfully via Jenkins!</h1>
11 Version 1.0 is live
12 </body>
13 </html>


krishmakhija24 / jenkins-demo-app
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
jenkins-demo-app / Jenkinsfile in main
Edit
Preview
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application...'
}
}
stage('Test') {
steps {
echo 'Testing the application...'
}
}
stage('Deploy') {
steps {
echo 'Deploying the application...'
bat 'if not exist "C:\\Deployments\\DemoApp" mkdir "C:\\Deployments\\DemoApp"'
bat 'copy index.html "C:\\Deployments\\DemoApp\\index.html"'
}
}
}
}


New Item
Enter an item name
GitHub-Jenkins-Demo
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.
OK


Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/krishmakhija24/jenkins-demo-app.git
Credentials
- none -
Advanced
Add Repository

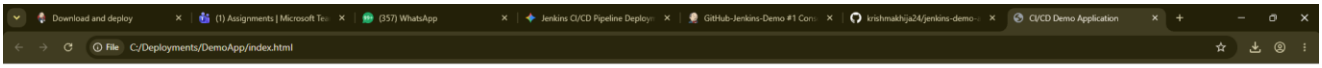

Jenkins / GitHub-Jenkins-Demo
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
GitHub-Jenkins-Demo
Permalinks


Jenkins / GitHub-Jenkins-Demo / #1
Status
Changes
Console Output
Delete build '#1'
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Console
Started by user Krish Makhija
Obtained Jenkinsfile from git https://github.com/krishmakhija24/jenkins-demo-app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/krishmakhija24/jenkins-demo-app.git
> git.exe init C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo # timeout=10
Fetching upstream changes from https://github.com/krishmakhija24/jenkins-demo-app.git
> git.exe --version # timeout=10
> git --version # 'git version 2.55.0.windows.3'
> git.exe fetch --tags --force --progress -- https://github.com/krishmakhija24/jenkins-demo-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/krishmakhija24/jenkins-demo-app.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision dab20ed0e5837c41f580773fcc9bb7511e6ce329 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f dab20ed0e5837c41f580773fcc9bb7511e6ce329 # timeout=10
Commit message: "Add Jenkinsfile for CI/CD pipeline setup"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)

```
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] bat

C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo>if not exist "C:\Deployments\DemoApp" mkdir "C:\Deployments\DemoApp"
[Pipeline] bat

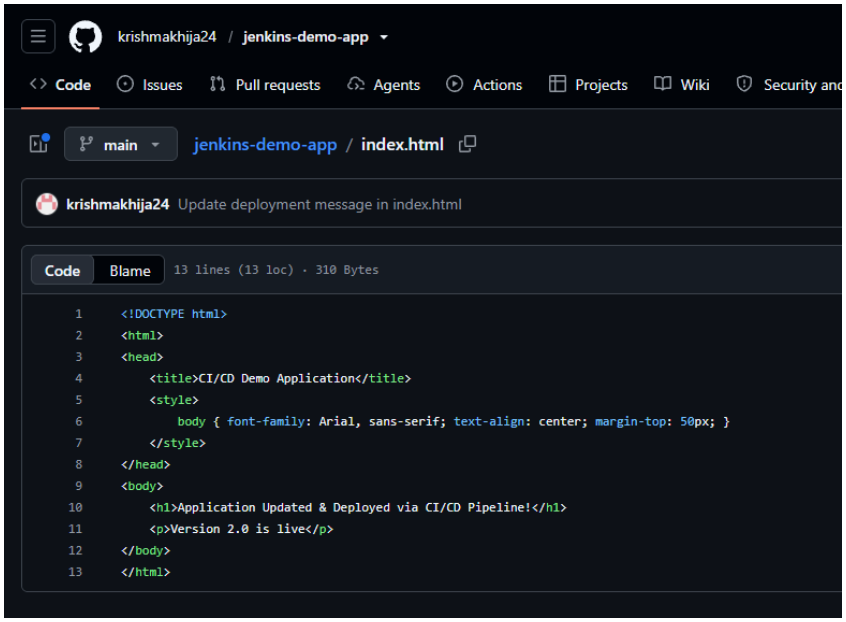
C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo>copy index.html "C:\Deployments\DemoApp\index.html"
        1 file(s) copied.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

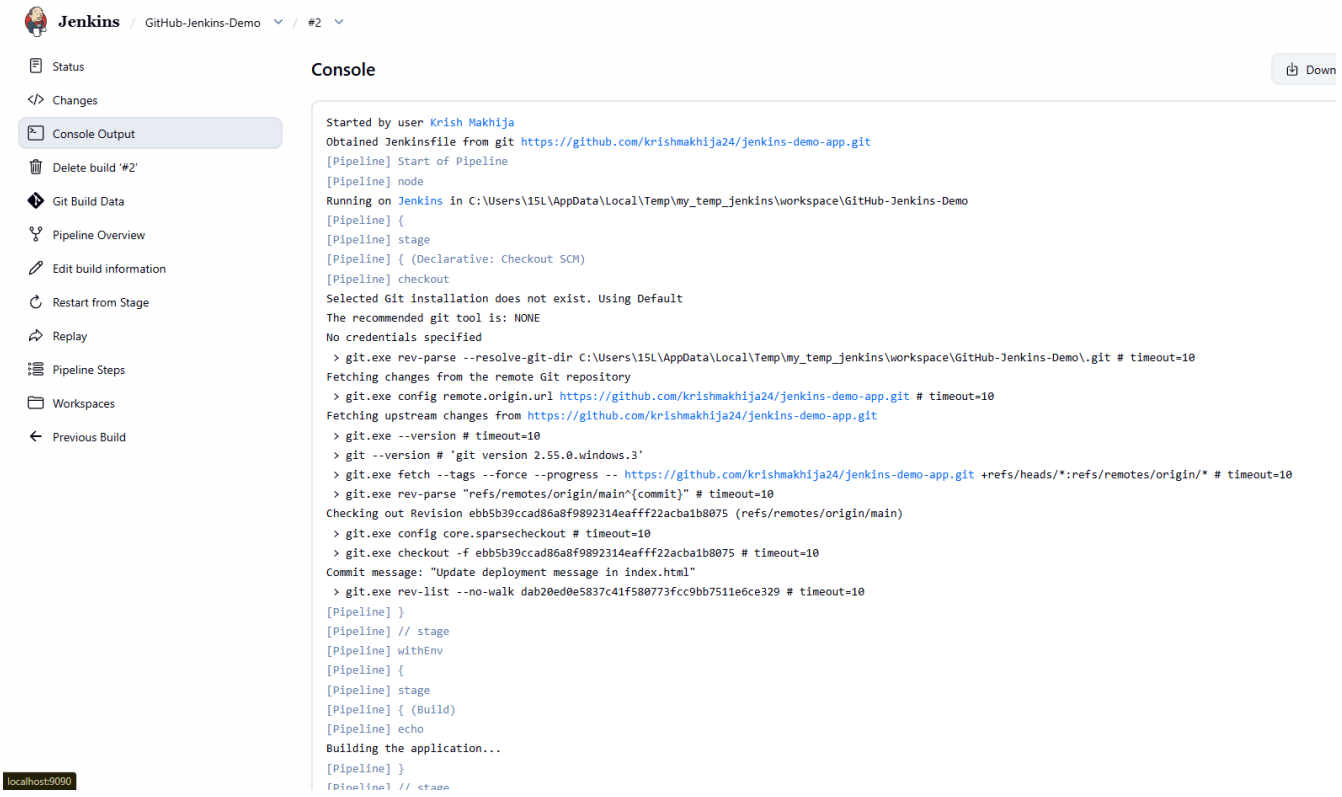


Application Deployed Successfully via Jenkins!

Version 1.0 is live




krishmakhija24 / jenkins-demo-app
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and
main
jenkins-demo-app / index.html
krishmakhija24 Update deployment message in index.html
Code
Blame
13 lines (13 loc) · 310 Bytes
<!DOCTYPE html>
<html>
<head>
<title>CI/CD Demo Application</title>
<style>
body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; }
</style>
</head>
<body>
<h1>Application Updated & Deployed via CI/CD Pipeline!</h1>
Version 2.0 is live
</body>
</html>


Jenkins / GitHub-Jenkins-Demo / #2
Status
Changes
Console Output
Delete build '#2'
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Down
Started by user Krish Makhija
Obtained Jenkinsfile from git https://github.com/krishmakhija24/jenkins-demo-app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/krishmakhija24/jenkins-demo-app.git # timeout=10
Fetching upstream changes from https://github.com/krishmakhija24/jenkins-demo-app.git
> git.exe --version # timeout=10
> git --version # 'git version 2.55.0.windows.3'
> git.exe fetch --tags --force --progress -- https://github.com/krishmakhija24/jenkins-demo-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision ebb5b39ccad86a8f9892314eafff22acba1b8075 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f ebb5b39ccad86a8f9892314eafff22acba1b8075 # timeout=10
Commit message: "Update deployment message in index.html"
> git.exe rev-list --no-walk dab20ed0e5837c41f580773fcc9bb7511e6ce329 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
[Pipeline] }
localhost:9090

```
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] bat

C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo>if not exist "C:\Deployments\DemoApp" mkdir "C:\Deployments\DemoApp"
[Pipeline] bat

C:\Users\15L\AppData\Local\Temp\my_temp_jenkins\workspace\GitHub-Jenkins-Demo>copy index.html "C:\Deployments\DemoApp\index.html"
       1 file(s) copied.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Conclusion:

The experiment successfully demonstrated an automated CI/CD pipeline using Jenkins and GitHub. Changes committed to the GitHub repository were automatically fetched, processed through structured declarative pipeline stages (Build, Test, Deploy), and deployed to the local deployment directory.

Utilizing a version-controlled Jenkinsfile ensured consistency, repeatability, and transparency throughout the delivery cycle. The seamless deployment and subsequent live update of the web application confirmed that CI/CD automation minimizes manual configuration errors, accelerates deployment speed, and establishes reliable software delivery.

LO3 mapped successfully.